

NEXORA

EVENT MARKETPLACE

Smart Event Management Ecosystem for Sri Lanka

Software Engineering Project Proposal

Document Type	Project Proposal
Version	2.0 — Final
Date	May 11, 2026
Duration	12 Weeks
Methodology	Agile (Scrum)

Prepared By

Name	Role
Gushan	Project Manager / Backend Lead
Pabavee	Frontend Lead
Avindu	Backend Developer
Savini	Frontend Developer
Navoda	QA & Testing Lead
Thejan	Database & DevOps Engineer

Submitted To: Academic Project Evaluation Panel

Document Control

Version	Date	Author	Description
0.1	2026-04-28	Gushan	Initial draft — EventHub concept
1.0	2026-05-05	Team	Revised scope, renamed to Nexora, roles updated
1.5	2026-05-10	Team	PostgreSQL specified, escrow clarified, timeline rebalanced
2.0	2026-05-11	Team	Final release — all sections reviewed and finalized

Table of Contents

1	Project Title
2	Introduction
3	Problem Statement
4	Objectives
5	Scope
6	Proposed System
7	Functional Requirements
8	Non-Functional Requirements
9	Technology Stack
10	Database Design
11	Project Timeline
12	Team Structure & Responsibilities
13	Required Project Documents
14	Risk Assessment & Mitigation
15	Expected Outcomes
16	Conclusion & Future Roadmap
17	Glossary
18	References

1 Project Title

Nexora Event Marketplace

A web-based event management ecosystem and multi-vendor marketplace designed for the Sri Lankan market. Nexora connects event organizers with verified service vendors — photographers, decorators, caterers, production teams, DJs, and more — through a single trusted platform.

2 Introduction

2.1 Background

Organizing an event in Sri Lanka typically requires searching across multiple platforms, making numerous phone calls, and relying on word-of-mouth to find and book vendors. This is time-consuming and often leads to poor choices due to a lack of verified information. No centralized digital platform currently addresses this gap.

2.2 Purpose

Nexora is built on three core pillars:

Unified Marketplace — Find, compare, and book all event vendors in one place.

Operational Management — Budget tracking, event checklists, and booking coordination.

Secure Financial Settlement — Escrow-based payments protecting both organizers and vendors.

2.3 Document Overview

This proposal covers the complete plan for developing Nexora using **Next.js** (frontend), **Node.js + Express** (backend), and **PostgreSQL + Prisma ORM** (database). It includes problem analysis, objectives, system design, tech stack, timeline, and team structure.

3 Problem Statement

#	Problem	Impact
P1	Unverified vendor information	No reliable way to check vendor credentials before booking.
P2	Lack of trust between parties	No standardized review or rating system exists.
P3	Fragmented event coordination	Planning across many vendors is stressful and error-prone.
P4	Unsafe payment methods	Informal cash/bank transfers with no protection for either party.
P5	No centralized platform	No single place to search, compare, and book event services.

Summary: There is no centralized, trusted, and secure platform connecting Sri Lankan event organizers with qualified vendors — resulting in wasted time, financial risk, and poor outcomes.

4 Objectives

4.1 Main Objective

To design and develop a secure, scalable web platform for event management and vendor booking in Sri Lanka using Next.js, Node.js, and PostgreSQL — improving trust, simplifying planning, and protecting financial transactions for all stakeholders.

4.2 Specific Objectives

ID	Objective	Linked Problem
SO 1	Improve vendor discovery through advanced search and filters (category, location, price, rating, availability).	P5
SO 2	Reduce planning stress with integrated budget tracking and booking coordination tools.	P3
SO 3	Provide secure payment handling through a simulated escrow workflow (sandbox integration for MVP).	P4
SO 4	Increase trust using verified vendor reviews and admin-approved vendor listings.	P1, P2
SO 5	Ensure full compliance with the Sri Lanka Personal Data Protection Act (PDPA).	—
SO 6	Deliver a working MVP within 12 weeks using Agile sprints.	—

5 Scope

5.1 In Scope — MVP Features

#	Feature	Description
1	Customer registration & login	Secure auth with role-based access control (RBAC).
2	Vendor search & filtering	Search by category, location, price, availability, and rating.
3	Vendor profile viewing	Services, pricing, portfolio, availability calendar, reviews.
4	Booking request & status tracking	Submit requests with date, location, service type, and notes.
5	Vendor dashboard	Manage profile, services, pricing, availability, and bookings.
6	Vendor booking management	Accept, reject, and track all booking requests.
7	Admin panel	Approve vendors, manage users, monitor activity, generate reports.

#	Feature	Description
8	Reviews & ratings	Star ratings and written reviews after service completion.
9	Budget tracking	Auto-calculated budget summary with category breakdown.
10	Escrow payment (simulated)	Simulated escrow status tracking with sandbox gateway.
11	PDPA-compliant data handling	Personal data handled per Sri Lanka data protection regulations.

5.2 Out of Scope (Deferred)

#	Excluded Feature	Reason
1	Video conferencing / live-streaming	Not core to marketplace functionality.
2	AI-based vendor recommendations	Requires ML infrastructure — deferred to Phase 3.
3	Vendor payroll, contracts, or tax filing	Outside platform scope.
4	Native iOS/Android apps	Web app is mobile-responsive; native apps deferred to Phase 4.
5	Real-time GPS tracking	Not essential for MVP booking flow.
6	Multi-user collaborative planning	Complex feature deferred to post-MVP.
7	Live payment processing	MVP uses simulated escrow; production payments in Phase 2.

5.3 MVP Definition

1. User registration and login (Customer, Vendor, Admin roles).
2. Vendor listing, profile viewing, and advanced search filters.
3. Booking request system with vendor accept/reject workflow.
4. Vendor booking management dashboard.
5. Admin vendor approval workflow.
6. Reviews and ratings module.
7. Budget tracking (auto-calculated from bookings).
8. Simulated escrow payment status tracking.

6 Proposed System

6.1 System Overview

Nexora is a fully web-based platform using **Next.js** (frontend), **Node.js + Express** (backend), and **PostgreSQL + Prisma ORM** (database). All interfaces — customer portal, vendor dashboard, and admin panel — are responsive web pages accessible from any modern browser.

6.2 User Roles

Role	Description	Key Capabilities
Customer	Event organizers searching for and booking vendors.	Register, search, view profiles, submit bookings, track budget, leave reviews.
Vendor	Service providers offering event-related services.	Register (pending approval), manage profile/services/pricing, accept/reject bookings.
Admin	Platform administrators overseeing operations.	Approve vendors, manage users, monitor activity, generate reports, handle disputes.

6.3 System Flow

1	Customer registers / logs in
	↓
2	Vendor registers / awaits admin approval
	↓
3	Admin reviews and approves vendor
	↓
4	Vendor profile becomes active on platform
	↓
5	Customer searches vendors (category, location, price, rating)
	↓
6	Customer views vendor profile and portfolio
	↓
7	Customer sends booking request (date, location, service, notes)
	↓
8	Vendor receives notification — accepts or rejects
	↓
9	Booking confirmed — escrow payment initiated (simulated)
	↓
10	Service delivered
	↓
11	Customer confirms completion — payment released to vendor
	↓

6.4 System Modules

Module	User	Description
Customer Portal	Customer	Search vendors, submit bookings, track status, manage budget, submit reviews.
Vendor Dashboard	Vendor	Manage profile/services/availability, accept or reject bookings, view feedback.
Admin Panel	Admin	Approve vendors, manage users, view reports, handle disputes.
Backend API	System	Node.js/Express REST APIs — auth (JWT), booking logic, escrow workflow, notifications.
Database	System	PostgreSQL + Prisma ORM. Stores users, vendors, bookings, reviews, payments.

6.5 System Architecture

CLIENT (Browser) – Next.js 14 (React / SSR / SSG)

Customer Portal | Vendor Dashboard | Admin Panel

↕ HTTPS / REST API

SERVER – Node.js + Express.js

Auth (JWT) | Booking Logic | Escrow Payment | Notifications

↕ Prisma ORM

DATABASE – PostgreSQL

Users | Vendors | Bookings | Reviews | Payments | Notifications

DEPLOYMENT: Vercel (Frontend) + Render (Backend + DB)

7 Functional Requirements

ID	Requirement	Description	Priority
FR01	User Registration & Login	JWT auth + RBAC. Passwords hashed with bcrypt.	High
FR02	Vendor Profile Management	Services, pricing, portfolio, availability calendar. JSONB for flexibility.	High
FR03	Vendor Discovery & Search	Filter by category, location, price, availability, rating. Full-text search via PostgreSQL.	High
FR04	Booking Request	Submit requests with event date, location, service type, notes. Availability validated.	High
FR05	Booking Management	Vendors accept/reject/track requests. Status: Pending → Accepted/Rejected → Completed.	High
FR06	Reviews & Ratings	Star ratings (1–5) and written reviews after service completion. Admin moderation.	Medium
FR07	Admin Vendor Approval	Admin must approve vendors before they go live. Can suspend/deactivate vendors.	High
FR08	Escrow Payment Workflow	Simulated: Customer pays → "held" → service done → "released" to vendor.	Medium
FR09	Budget Tracking	Auto-calculated total and category-wise cost summary from booked services.	Low
FR10	Reporting & Analytics	Admin reports: bookings, users, vendor status, revenue, activity metrics.	Medium

7.1 Traceability Matrix

Requirement	Objective	Problem
FR01	SO1, SO5	P1
FR02	SO1	P1, P5
FR03	SO1	P3, P5
FR04	SO2	P3
FR05	SO2	P3
FR06	SO4	P2
FR07	SO4	P1, P2
FR08	SO3	P4
FR09	SO2	P3

Requirement	Objective	Problem
FR10	SO4	—

8 Non-Functional Requirements

Category	ID	Requirement
Performance	NFR01	Core pages load within 3 seconds under normal load (up to 100 concurrent users).
Security	NFR02	Passwords hashed with bcrypt. JWT + RBAC on all endpoints. HTTPS enforced.
Usability	NFR03	Fully responsive (desktop, tablet, mobile). WCAG 2.1 Level A guidelines followed.
Scalability	NFR04	Backend supports horizontal scaling. PostgreSQL indexed for high-read operations.
Reliability	NFR05	Graceful error handling. Booking/payment data protected via database transactions.
Maintainability	NFR06	Modular architecture (controllers/services/routes/models). JSDoc comments throughout.
Compatibility	NFR07	Tested on Chrome, Firefox, Edge, and Safari (latest two versions).
Data Privacy	NFR08	PDPA compliant. User consent collected; data deletion requests supported.

9 Technology Stack

Layer	Technology	Purpose
Frontend	Next.js 14 (React)	Customer portal, vendor dashboard, and admin panel — SSR/SSG support.
Styling	Tailwind CSS	Utility-first framework for rapid, consistent, responsive UI.
State Management	Zustand	Lightweight client-side state management.
Backend	Node.js + Express.js	REST API server — auth, booking logic, payment workflow, business rules.
Database	PostgreSQL	Primary relational DB. JSONB used for flexible vendor metadata.
ORM	Prisma	Type-safe DB client. Schema migrations, queries, relationships.
Auth	JWT + bcrypt	Stateless sessions (JWT) and secure password hashing (bcrypt).
File Storage	Cloudinary	Vendor portfolio images and profile photos.
API Docs	Swagger / OpenAPI	Auto-generated interactive API documentation.
UI Design	Figma	Wireframes, prototypes, and UI components.
Version Control	GitHub	Source code management, branching strategy, team collaboration.
Testing	Jest, Supertest, Cypress	Unit, API integration, and end-to-end browser testing.
Deployment	Vercel + Render	Vercel for Next.js; Render for API server and PostgreSQL.

10 Database Design

10.1 Why PostgreSQL?

Criterion	Justification
Relational integrity	Foreign keys, constraints, ACID transactions — critical for booking/payment consistency.
JSONB support	Flexible storage of vendor metadata without needing a separate NoSQL database.
Full-text search	Built-in vendor search — no external search engine needed for MVP.
Prisma compatibility	First-class support with type-safe queries and auto-generated migrations.
Scalability	Indexing, query optimization, and connection pooling for production workloads.

10.2 Core Tables

Table	Description	Key Fields
users	All registered users	id, email, password_hash, role, name, phone, is_active, created_at
vendor_profiles	Extended vendor info	id, user_id (FK), business_name, category, location, portfolio (JSONB), is_approved
services	Vendor service listings	id, vendor_id (FK), name, price, duration, images (JSONB), is_active
bookings	Booking requests	id, customer_id, vendor_id, service_id, event_date, status (enum), notes
payments	Escrow payment records	id, booking_id (FK), amount, status (enum: pending/held/released/refunded)
reviews	Customer reviews	id, customer_id, vendor_id, booking_id, rating (1–5), comment, created_at
notifications	System notifications	id, user_id (FK), type, message, is_read, created_at

10.3 Entity Relationships

users (1) [REDACTED] (1) vendor_profiles
users (1) [REDACTED] (N) bookings [as customer]
vendor_profiles (1) [REDACTED] (N) services
vendor_profiles (1) [REDACTED] (N) bookings [as vendor]
bookings (1) [REDACTED] (1) payments
bookings (1) [REDACTED] (0..1) reviews
users (1) [REDACTED] (N) notifications

11 Project Timeline

Agile (Scrum) — 6 sprints over 12 weeks (2-week sprints).

Sprint	Weeks	Phase	Key Deliverables
Sprint 0	1–2	Planning & Requirements	SRS document, use case diagrams, sequence diagrams, GitHub setup, branching strategy.
Sprint 1	3–4	Design & Architecture	Figma wireframes, PostgreSQL schema + Prisma, REST API design, ER diagram, class diagram.
Sprint 2	5–6	Core Backend + Search	Auth APIs (JWT + bcrypt), vendor CRUD APIs, booking APIs, search/filter APIs — tested with Postman.
Sprint 3	7–8	Frontend + Reviews	Next.js connected to APIs. Vendor listing, profile, booking flow, dashboards, reviews module.
Sprint 4	9–10	Admin + Payments + PDPA	Admin panel, escrow payment workflow (simulated), PDPA compliance, budget tracking.
Sprint 5	11	Testing & Hardening	End-to-end testing (Cypress), API tests (Jest/Supertest), bug fixes, security review, cross-browser testing.
Sprint 6	12	Deployment & Demo	Final documentation, user manual, deployment to Vercel + Render, demo presentation.

Each sprint ends with a **Sprint Review** and **Retrospective** to assess progress and adjust the backlog.

12 Team Structure & Responsibilities

12.1 Sub-Teams

Team	Members	Focus
Backend & Infrastructure	Gushan, Avindu, Thejan	API server, PostgreSQL schema, REST APIs, deployment pipeline.
Frontend & Quality	Pabavee, Savini, Navoda	UI/UX design, portals, testing, documentation, final report.

12.2 Individual Roles

Name	Role	Responsibilities
Gushan	Project Manager + Backend Lead	Project planning, backend architecture, API design, documentation oversight.
Avindu	Backend Developer	REST APIs, authentication (JWT + bcrypt), booking logic, escrow workflow.
Thejan	Database & DevOps	PostgreSQL design, Prisma schema/migrations, deployment (Vercel + Render), GitHub.

Name	Role	Responsibilities
Pabavee	Frontend Lead	UI/UX (Figma), customer portal, vendor profile pages, frontend architecture.
Savini	Frontend Developer	Admin panel UI, vendor dashboard UI, component library, search/filter interface.
Navoda	QA & Testing Lead	Test cases, Cypress testing, bug reports, cross-browser QA, user manual.

13 Required Project Documents

#	Document	Description	Responsible
1	Software Requirements Specification (SRS)	Complete functional and non-functional requirements with traceability.	Gushan
2	Use Case Diagram	Visual representation of system actors and their interactions.	Team
3	Entity-Relationship (ER) Diagram	PostgreSQL database schema and table relationships.	Thejan
4	Class Diagram	Object-oriented design of backend services and models.	Avindu
5	Sequence Diagram	Interaction flow for key processes (booking, payment, review).	Gushan
6	Activity Diagram	Workflow diagrams for customer booking and vendor approval processes.	Team
7	Data Flow Diagram	Visual representation of data movement through the system.	Savini
8	System Architecture Diagram	High-level architecture showing frontend, backend, database, deployment.	Gushan, Thejan
9	API Documentation (Swagger)	Auto-generated REST API documentation with request/response examples.	Avindu
10	Test Cases & Bug Reports	Comprehensive test cases for all modules with tracked bug reports.	Navoda
11	Final Project Report	Complete project documentation with methodology, implementation, results.	Team
12	User Manual	End-user guide for customers, vendors, and administrators.	Navoda, Pabavee

14 Risk Assessment & Mitigation

#	Risk	Likelihood	Impact	Mitigation
R 1	Scope creep	High	High	Strict MVP scope in Section 5. New features deferred to post-MVP backlog.
R 2	Timeline delays	Medium	High	Agile sprints + bi-weekly reviews. Sprint 5 buffer for catch-up.
R 3	Team unavailability	Medium	Medium	Cross-training between sub-teams. All code documented on GitHub.
R 4	Database performance issues	Low	Medium	PostgreSQL indexing, query optimization, and Prisma profiling from Sprint 2.
R 5	Third-party service downtime	Low	Medium	Local file upload fallback. Deployment tested on multiple platforms.
R 6	Security vulnerabilities	Medium	High	OWASP top-10 checklist. JWT expiry, input validation, Prisma prevents SQL injection.
R 7	PDPA compliance gaps	Low	High	Data policies defined in Sprint 0. Consent forms and deletion APIs in Sprint 4.

15 Expected Outcomes

Upon successful completion, the following outcomes are expected:

1. A **fully functional, demo-ready** Nexora web platform connecting organizers and vendors in Sri Lanka.
2. **Reduced time and effort** for customers booking event services — from days to minutes.
3. **Improved trust** through verified reviews, admin approval, and simulated secure escrow payments.
4. **Full PDPA compliance** ensuring user data privacy and legal protection.
5. A **clean, modular codebase** with clear potential for future phases.
6. **Comprehensive documentation** — SRS, UML diagrams, API docs, test reports, and user manual.

16 Conclusion & Future Roadmap

16.1 Conclusion

Nexora provides a modern, practical solution for the Sri Lankan event industry. Built with **Next.js**, **Node.js**, **PostgreSQL**, and **Prisma ORM**, the platform is secure, scalable, and PDPA compliant from day one. The team will deliver a fully functional MVP within 12 weeks using Agile methodology.

16.2 Future Roadmap

Phase	Enhancement	Description
Phase 2	Real Payment Gateway	Migrate from simulated escrow to production payments via PayHere or Stripe.
Phase 3	AI Vendor Recommendations	ML models using booking history and preferences for personalized suggestions.
Phase 4	Native Mobile Apps	iOS and Android apps using React Native.

17 Glossary

Term	Definition
API	Application Programming Interface — a set of protocols for building and integrating software.
CRUD	Create, Read, Update, Delete — the four basic database operations.
Escrow	A financial arrangement where a third party holds funds until transaction conditions are met.
JWT	JSON Web Token — open standard for securely transmitting information as a JSON object.
MVP	Minimum Viable Product — the simplest version of a product that delivers core value.
ORM	Object-Relational Mapping — converting data between incompatible type systems in OOP.
PDPA	Personal Data Protection Act — Sri Lanka's data privacy legislation.
RBAC	Role-Based Access Control — restricting system access based on user roles.
REST	Representational State Transfer — architectural style for designing networked applications.
SRS	Software Requirements Specification — document describing software purpose and requirements.
SSR/SSG	Server-Side Rendering / Static Site Generation — rendering strategies in Next.js.
WCAG	Web Content Accessibility Guidelines — international standards for web accessibility.

18 References

1. Next.js Documentation — <https://nextjs.org/docs>
2. Node.js Documentation — <https://nodejs.org/en/docs>
3. PostgreSQL Documentation — <https://www.postgresql.org/docs/>
4. Prisma ORM Documentation — <https://www.prisma.io/docs>
5. Express.js Documentation — <https://expressjs.com/>
6. Tailwind CSS Documentation — <https://tailwindcss.com/docs>
7. JWT Introduction — <https://jwt.io/introduction>
8. OWASP Top Ten — <https://owasp.org/www-project-top-ten/>
9. Sri Lanka Personal Data Protection Act (PDPA), No. 9 of 2022
10. WCAG 2.1 Guidelines — <https://www.w3.org/WAI/WCAG21/quickref/>
11. Agile Scrum Framework — <https://www.scrum.org/resources/what-scrum-module>

— End of Document —

Nexora Event Marketplace — Project Proposal v2.0

© 2026 Nexora Project Team. All rights reserved.

Prepared by: Gushan | Pabavee | Avindu | Savini | Navoda | Thejan